

Manipulation les registres et les masques

Avant de décrire le fonctionnement des différents périphériques et leur configuration, commençons par étudier les méthodes de programmation pour interagir avec eux.

Comprendre la notion de registre

Sur les microcontrôleurs les périphériques sont configurés et contrôlés à travers des registres ayant une adresse mémoire réservée. Cela signifie que l'accès et la configuration d'un périphérique se fait au niveau du code simplement en écrivant ou en lisant à des adresses mémoires spécifiques.

Un registre est généralement de la taille d'un mot du processeur, par exemple de 32 bits pour le STM32, et chaque bit qui le compose peut avoir une fonctionnalité différente. Prenons à titre d'exemple le registre CR1 qui est associé à un timer (vous verrez dans la suite ce qu'est un timer) sur un STM32F103, la documentation de ce microcontrôleur nous en fournit la description suivante :

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0															
0x00	TIMx_CR1	Reserved																						CKD [1:0]		ARPE	CMS [1:0]		DIR	OPM	URS	UDIS	CEN															
	0																							0	0		0	0						0														
	Reset value																																															

Documentation STM32F103 [ST, RM0008, DocID13902 Rev 16, nov. 2015, p. 422]

Ce tableau permet de savoir que les bits 10 à 31 du registre CR1 ne sont pas utilisés et que les bits 0 à 9 ont des fonctions bien particulières.

Toujours dans la même documentation, en consultant la description de ce registre (p. 403), nous apprenons que le bit 0, nommé CEN, sert à mettre en marche ou à stopper le compteur du timer, et que les bits 5 et 6, nommés CMS, permettent de configurer la manière dont le timer compte (en incrémentant, en décrémentant, etc.).

Cet exemple montre que les bits d'un registre doivent être considérés de manière indépendante et que pour configurer un périphérique il faut être capable de modifier un ou plusieurs bits sans modifier les autres. Pour réaliser cela, nous allons utiliser des **masques logiques** avec **des opérateurs bit-à-bit**.

Les opérateurs logiques en C

Commençons par étudier les opérateurs logiques bit-à-bit. Ils sont de quatre types : ET, OU, XOR (OU exclusif), NON. Nous retrouvons la même variété d'opérations que les opérateurs booléens classiques, mais au lieu de traiter l'opération en considérant la valeur des opérandes, les opérateurs bit-à-bit appliquent l'opération au niveau des bits des opérandes, d'où leur nom...

La syntaxe en langage C des opérateurs logiques bit-à-bit est :

Fonction logique	Opérateur logique	Opérateur bit-à-bit
ET	&&	&
OU		
XOR		^
NON	!	~

Rappelons qu'en C, les opérateurs logiques considèrent les opérandes comme des valeurs booléennes fausses si tous les bits sont nuls et vrai sinon. Ainsi une valeur VarA égale à 6, soit b0110 en binaire, est considérée comme étant vraie. La condition du test if dans l'exemple suivant est donc vraie, l'affichage « C'est vrai » se fera.

```
VarA = 6;

if (varA){

    printf("C'est vrai \n");

}
```

Manipulons maintenant les valeurs VarA et VarB suivantes

```
VarA = 6; // soit b0110 en binaire

VarB = 1; // soit b0001 en binaire
```

L'application de l'opérateur logique ET sur ces deux valeurs, VarA && VarB, donne 1 (vrai) car VarA et VarB sont vraies.

Par contre, les opérateurs bit-à-bit ET, OU, XOR et NON donnent les résultats suivants :

	b3	b2	b1	b0
a	0	1	1	0
b	0	1	0	1
a & b	0	1	0	0
a b	0	1	1	1
a ^ b	0	0	1	1
~a	1	0	0	1

Comme nous allons le voir dans la suite, c'est grâce à ces opérateurs bit-à-bit que vous allez pouvoir manipuler les valeurs dans les registres des périphériques.

Mettre un bit à 1

Pour mettre un seul bit à 1 dans un registre, nous utilisons l'opérateur bit-à-bit OU avec un masque dont tous les bits sont à 0, sauf celui que l'on veut initialiser.

Ainsi, pour une valeur de l'octet init décrite en binaire par b7 b6 b5 b4 b3 b2 b1 b0, si vous voulez passer le bit b4 à 1, vous allez composer un masque, mask, ayant la valeur binaire b0001 0000 et appliquer l'opérateur bit-à-bit OU. En C, cela s'écrit :

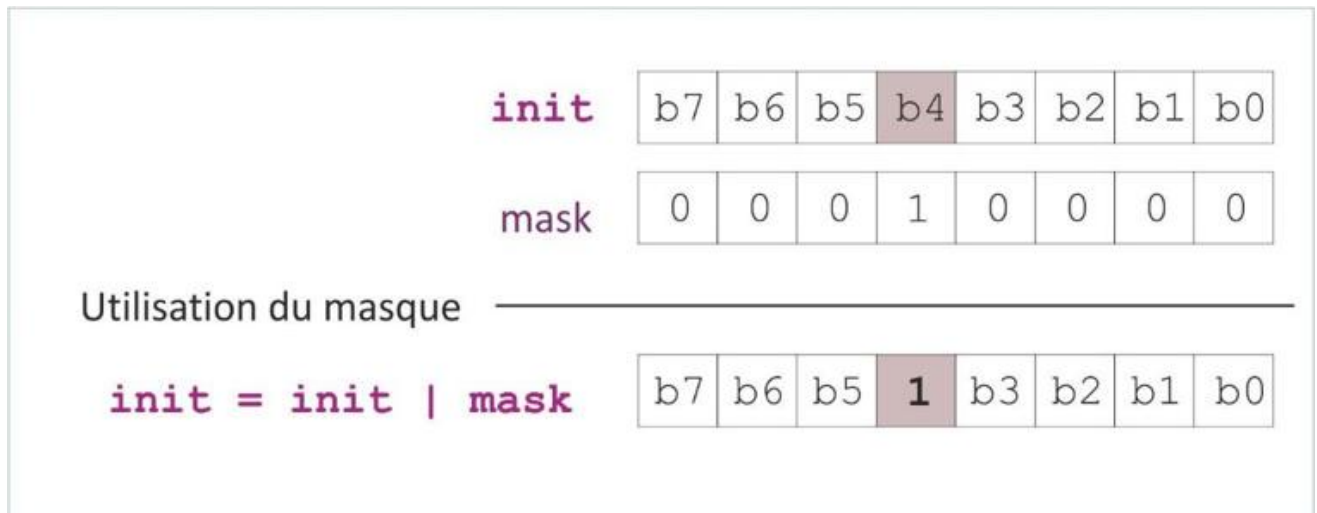
```
mask = 0x10; // valeur hexadécimale pour 0b0001 0000
```

```
init = init | mask; // application du masque à la valeur init
```

La dernière ligne peut aussi s'écrire plus succinctement :

```
init |= mask;
```

La valeur init aura pour valeur finale b7 b6 b5 **1** b3 b2 b1 b0. Ainsi le bit b4 est passé à 1 sans modifier la valeur des autres bits.



Passage du bit `b4` à 1

Vous avez dû remarquer que pour effectuer cette opération en langage C, il faut calculer la valeur du masque binaire en hexadécimal ou en décimal car le standard du langage C n'admet pas de littéraux en binaire. Ainsi, les opérations précédentes peuvent être écrites sans passer par la variable `mask`, soit :

```
init = init | 0x10; // opérateur avec la valeur hexadécimale
```

soit

```
init = init | 15; // opérateur identique avec la valeur décimale
```

Sauf si vous parlez couramment le binaire, il n'est pas évident de comprendre que `0x10` ou `15` correspond à un masque visant le bit de rang 4 et la relecture du code en devient compliquée. De plus, il est très facile de se tromper lorsque l'on fait la conversion soi-même.

Pour éviter les erreurs, il vaut mieux utiliser l'opérateur bit-à-bit `<<` de décalage à gauche. Par exemple, `1 << x` affecte 1 au bit de rang `x` et construit ainsi un masque. Les opérations précédentes s'écrivent alors

```
init = init | (1 << 4);
```

Forme qui permet de comprendre immédiatement que le bit 4 de `init` est mis à 1.

Mettre plusieurs bits à 1

Vous pouvez aussi utiliser cette méthode pour mettre à 1 plusieurs bits en même temps. Reprenons l'exemple précédent avec `init` une variable décrite par 8 bits `b7 b6 b5 b4 b3 b2 b1 b0` pour laquelle nous voulons passer les bits `b4`, `b1` et `b0` à 1. Une première solution est de faire une opération pour chaque bit

```
init = init | (1 << 4);
```

```
init = init | (1 << 1);
```

```
init = init | (1 << 0);
```

Une seconde solution plus efficace, est de construire un masque où les bits 0, 1 et 4 sont à 1 (b0001 0011 en binaire, 0x13 en hexadécimal, 19 en décimal), puis d'utiliser ce masque avec l'opérateur bit-à-bit OU :

```
init = init | 0x13; // masque avec la valeur b0001 0011
```

On obtient de cette manière l'affectation de la valeur b7 b6 b5 **1** b3 b2 **11** à init.

init	b7	b6	b5	b4	b3	b2	b1	b0
Masque poids 0 : (1 << 0)	0	0	0	0	0	0	0	1
Masque poids 1 : (1 << 1)	0	0	0	0	0	0	1	0
Masque poids 4 : (1 << 4)	0	0	0	1	0	0	0	0
Masque complet (combinaison ou)								
(1 << 0) (1 << 1) (1 << 4)	0	0	0	1	0	0	1	1
Utilisation du masque complet								
init = init ((1 << 0) (1 << 1) (1 << 4))	b7	b6	b5	1	b3	b2	1	1

Affectation de la valeur b7 b6 b5 **1** b3 b2 **11** à init

Au lieu de faire vous-même le calcul du masque complet, vous pouvez aussi combiner les masques unitaires à l'aide de l'opérateur bit-à-bit OU, soit pour le cas précédent :

```
init = init | ((1<<0) | (1<<1) | (1<<4));
```

Le compilateur se chargera de calculer le masque à votre place ce qui évitera bien des erreurs. De plus, cette manière d'écrire les masques facilite grandement la relecture du code.

Mettre des bits à 0

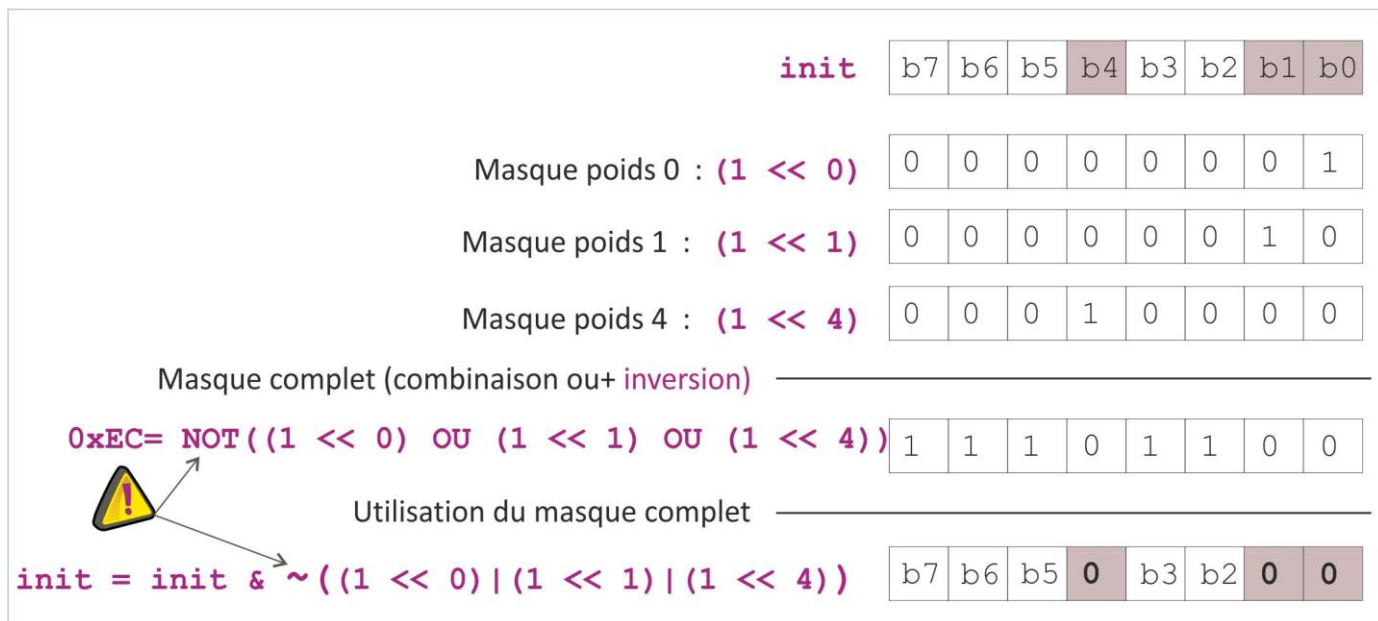
Pour mettre des bits à 0, nous utilisons l'opérateur bit-à-bit ET avec un masque ayant des bits à 0 uniquement devant les bits que nous voulons initialiser. Ainsi, si vous souhaitez passer à 0 les bits b4, b1 et b0 d'un octet init dont la valeur en binaire est décrite par b7 b6 b5 b4 b3 b2 b1 b0, vous utiliserez un masque binaire b1110 1100 avec l'opérateur ET, ce qui s'écrit

```
init = init & 0xEC; // 0xEC est la valeur hexadécimale pour b1110 1100
```

ou de manière équivalente

```
init = init & 0xEC;
```

ce qui fixe la valeur de init à b7 b6 b5 **0** b3 b2 **00** :



Passer la valeur de init à b7 b6 b5 0 b3 b2 0 0

Attention dans cet exemple le mot est de 8 bits, si les mots étaient de 32 bits il faudrait compléter le masque avec des 1 pour couvrir les 32 bits, soit la valeur 0xFFFFFEC. Une manière beaucoup plus simple, et surtout plus fiable, est de construire le masque avec des 1 pour les bits à annuler et ensuite d'inverser la valeur avec l'opérateur bit-à-bit NON. Ainsi l'écriture du masque de l'exemple précédent devient

```
init = init & ~ ( (1<<0)|(1<<1)|(1<<4) );
```

Avec cette écriture, il est facile d'identifier rapidement que ce sont les bits 0, 1 et 4 qui sont mis à 0.

Il est impossible de mettre à 0 et à 1 des bits en une seule opération. Il faut toujours passer par deux opérations ; l'une pour mettre les bits à 0 et une pour ceux à 1.

Tester un bit

Les opérateurs bit-à-bit peuvent aussi servir pour tester la valeur d'un bit dans un registre. Pour cela, il faut créer un masque en mettant à 0 tous les bits qui n'ont pas d'intérêt pour le test à faire, puis l'utiliser avec l'opérateur bit-à-bit ET.

Prenons un exemple : supposons que vous avez une valeur value égale à b7 b6 b5 b4 b3 b2 b1 b0 et que vous voulez tester si le bit 5 est à 1.

Commencez par préparer un masque avec des 0 partout sauf pour le bit 5, soit

```
mask = (1 << 5);
```

et appliquez le à la valeur value avec l'opérateur bit-à-bit ET :

```
value = value & mask;
```

Vous obtenez comme résultat 0 0 b5 0 0 0 0 0, ce qui représente une valeur équivalente à vraie si b5 vaut 1 et faux si b5 vaut 0.

Vous pouvez ainsi utiliser le résultat de l'opération directement comme condition d'un test. Par exemple

```
if ( value & (1 << 5) ) {  
  
    ...  
  
}
```

permet de tester si le bit 5 de value est à 1 et

```
if !( value & (1 << 5) ) {  
  
    ...  
  
}
```

s'il est à 0.

À la fin de ce chapitre, vous êtes capable :

- d'utiliser les opérateurs bits-à-bits en C,
- d'expliquer et de mettre en pratique un masque pour manipuler les bits d'un registre.